

# Sprint 5 Structured Notes: Functions, Scope, and Functional Thinking

## 0. What Sprint 5 Is About

Sprint 5 is about learning how to write reusable JavaScript code.

In earlier sprints, you learned how to work with:

- values
- variables
- data types
- strings
- numbers
- booleans
- operators
- conditions

Now Sprint 5 teaches you how to put logic into **functions**.

A function lets you write code once and use it many times.

Main goal:

Write reusable code and keep behavior predictable.

By the end of Sprint 5, you should understand:

- what functions are
- why functions are useful
- function declarations
- function calls / invocation
- parameters
- arguments
- return values
- what happens when there is no `return`
- `return` vs `console.log()`
- default parameters
- anonymous functions
- function expressions
- arrow functions
- implicit return
- scope
- global scope

- local / function scope
  - block scope
  - pure functions
  - side effects
  - separating calculation from side effects
  - callbacks at a beginner level
  - higher-order functions at a beginner level
  - function factories
  - currying at a beginner level
  - closures
  - private state with closures
  - common beginner mistakes with functions
- 

## Part A: The Core Idea

### 1. What Is a Function?

A **function** is a reusable block of code.

Simple meaning:

A function is code you can save, name, and run later.

Example:

```
function sayHello() {  
  return "Hello";  
}
```

This creates a function named `sayHello`.

To run the function, you call it:

```
console.log(sayHello());
```

Expected output:

```
Hello
```

---

## 2. Why Functions Are Useful

Functions help you avoid repeating code.

Without a function:

```
const price1 = 10;
const quantity1 = 3;
const total1 = price1 * quantity1;
console.log(total1);

const price2 = 5;
const quantity2 = 4;
const total2 = price2 * quantity2;
console.log(total2);
```

This works, but the logic is repeated.

With a function:

```
function getTotal(price, quantity) {
  return price * quantity;
}

console.log(getTotal(10, 3));
console.log(getTotal(5, 4));
```

Expected output:

```
30
20
```

The function lets you reuse the same calculation with different values.

---

## 3. Function Mental Model

Think of a function like a small machine.

```
input → function → output
```

Example:

10 and 3 → getTotal → 30

In code:

```
function getTotal(price, quantity) {  
  return price * quantity;  
}  
  
console.log(getTotal(10, 3));
```

10 and 3 go in.

The function multiplies them.

30 comes out.

---

## Part B: Function Declaration and Invocation

### 4. Function Declaration

A **function declaration** creates a named function.

Syntax:

```
function functionName() {  
  // code here  
}
```

Example:

```
function greet() {  
  return "Hello";  
}
```

This defines the function.

But defining a function does not run it yet.

---

## 5. Function Invocation / Calling a Function

To run a function, write its name followed by parentheses.

```
greet();
```

This is called:

- calling a function
- invoking a function
- running a function

If the function returns a value, you can print it:

```
function greet() {  
  return "Hello";  
}  
  
console.log(greet());
```

Expected output:

```
Hello
```

---

## 6. Defining vs Calling

These are different:

```
function greet() {  
  return "Hello";  
}
```

This only creates the function.

This runs it:

```
greet();
```

This runs it and prints the result:

```
console.log(greet());
```

Beginner rule:

A function does nothing until it is called.

---

## Part C: Parameters and Arguments

### 7. What Is a Parameter?

A **parameter** is a placeholder inside a function definition.

Example:

```
function greet(name) {  
  return `Hello, ${name}`;  
}
```

Here, `name` is a parameter.

It is not a real value yet.

It is a placeholder for a value that will be provided later.

---

### 8. What Is an Argument?

An **argument** is the actual value passed into the function when you call it.

Example:

```
function greet(name) {  
  return `Hello, ${name}`;  
}  
  
console.log(greet("Ada"));
```

Here:

name

is the parameter.

"Ada"

is the argument.

Expected output:

Hello, Ada

## 9. Parameter vs Argument

Simple comparison:

| Word      | Meaning                                  | Example |
|-----------|------------------------------------------|---------|
| Parameter | Placeholder in the function definition   | name    |
| Argument  | Real value passed into the function call | "Ada"   |

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3));
```

Here:

- `a` and `b` are parameters
- `2` and `3` are arguments

Expected output:

5

## 10. Multiple Parameters

A function can have more than one parameter.

Example:

```
function formatFullName(firstName, lastName) {  
  return `${firstName} ${lastName}`;  
}  
  
console.log(formatFullName("Ada", "Lovelace"));
```

Expected output:

```
Ada Lovelace
```

The first argument goes into the first parameter.

The second argument goes into the second parameter.

```
"Ada" → firstName  
"Lovelace" → lastName
```

---

## Part D: Return Values

### 11. What Does `return` Do?

`return` sends a value back from a function.

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(2, 3);  
console.log(result);
```

Expected output:



5

Simple meaning:

`return` is how a function gives an answer back.

---

## 12. `return` Stops the Function

When JavaScript reaches `return`, the function stops.

Example:

```
function checkAge(age) {  
  if (age >= 18) {  
    return "Adult";  
  }  
  
  return "Minor";  
}  
  
console.log(checkAge(20));  
console.log(checkAge(15));
```

Expected output:

```
Adult  
Minor
```

If `age >= 18` is true, JavaScript returns `"Adult"` and stops the function.

It does not continue to the next return.

---

## 13. What Happens If There Is No `return`?

If a function does not return anything, JavaScript returns `undefined`.

Example:

```
function sayHi() {  
  const message = "Hi";  
}  
  
console.log(sayHi());
```

Expected output:

undefined

Why?

Because the function created a variable, but it did not return it.

Correct version:

```
function sayHi() {  
  const message = "Hi";  
  return message;  
}  
  
console.log(sayHi());
```

Expected output:

Hi

Beginner rule:

If you want a function to give a value back, use `return`.

---

## Part E: `return` vs `console.log()`

### 14. `console.log()` Prints a Value

`console.log()` prints something in the browser console.

Example:

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(2, 3);
```

Expected output:

5

This prints 5, but the function does not give 5 back as a reusable result.

---

## 15. return Gives a Reusable Value

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(2, 3);  
const doubled = result * 2;  
  
console.log(doubled);
```

Expected output:

10

This works because add(2, 3) returned 5.

Then 5 was reused in another calculation.

---

## 16. Beginner Rule for return and console.log()

Use return when the function should produce a value.

Use console.log() when you want to inspect or display something while learning.

Better function style:

```
function getTotal(price, quantity) {  
  return price * quantity;  
}  
  
const total = getTotal(10, 3);  
console.log(total);
```

The function calculates.

The `console.log()` displays.

This separation makes the code cleaner.

---

## Part F: Default Parameters

### 17. What Is a Default Parameter?

A default parameter is a fallback value.

It is used when no argument is provided.

Example:

```
function greet(name = "Guest") {  
  return `Hello, ${name}!`;  
}  
  
console.log(greet("Anna"));  
console.log(greet());
```

Expected output:

```
Hello, Anna!  
Hello, Guest!
```

When `"Anna"` is passed, JavaScript uses `"Anna"`.

When no argument is passed, JavaScript uses `"Guest"`.

---

## 18. Why Default Parameters Are Useful

Default parameters make functions safer and friendlier.

Example without a default:

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
console.log(greet());
```

Expected output:

```
Hello, undefined!
```

Better version:

```
function greet(name = "Guest") {  
  return `Hello, ${name}!`;  
}  
  
console.log(greet());
```

Expected output:

```
Hello, Guest!
```

---

## Part G: Function Expressions and Anonymous Functions

### 19. Function Declaration vs Function Expression

A function declaration looks like this:

```
function add(a, b) {  
  return a + b;  
}
```

A function expression stores a function in a variable:

```
const add = function(a, b) {  
  return a + b;  
};
```

Both can be called like this:

```
console.log(add(2, 3));
```

Expected output:

```
5
```

---

## 20. Anonymous Function

An anonymous function is a function without a name.

Example:

```
const add = function(a, b) {  
  return a + b;  
};
```

This function itself has no name after the `function` keyword.

But it is stored in a variable named `add`.

So you can still call it:

```
console.log(add(2, 3));
```

---

## Part H: Arrow Functions

### 21. What Is an Arrow Function?

An arrow function is a shorter way to write a function.

Normal function:

```
function add(a, b) {  
  return a + b;  
}
```

Arrow function:

```
const add = (a, b) => {  
  return a + b;  
};
```

Both return the same result.

```
console.log(add(2, 3));
```

Expected output:

```
5
```

---

### 22. Arrow Function with Implicit Return

If the arrow function has one expression, you can often remove `{ }` and `return`.

```
const add = (a, b) => a + b;  
  
console.log(add(2, 3));
```

Expected output:

5

This is called an **implicit return**.

Simple meaning:

JavaScript returns the expression automatically.

---

## 23. Arrow Function Beginner Rule

Use normal function syntax while learning the concept:

```
function add(a, b) {  
  return a + b;  
}
```

Use arrow functions for short functions after you understand `return`:

```
const add = (a, b) => a + b;
```

Do not rush arrow functions if normal functions are clearer.

---

# Part I: Scope

## 24. What Is Scope?

Scope means where a variable can be used.

Simple meaning:

Scope controls where names exist in your code.

If a variable is outside your current scope, you cannot use it.

---

## 25. Global Scope

A variable in global scope is available broadly.



Example:

```
const appName = "Study App";

function showAppName() {
  return appName;
}

console.log(showAppName());
```

Expected output:

Study App

`appName` is outside the function.

The function can read it.

But too many global variables can make programs messy.

Beginner rule:

Do not make everything global.

---

## 26. Local / Function Scope

A variable created inside a function only exists inside that function.

Example:

```
function createMessage() {
  const message = "Hello";
  return message;
}

console.log(createMessage());
```

Expected output:

Hello

But this does not work:

```
function createMessage() {  
  const message = "Hello";  
  return message;  
}  
  
console.log(message);
```

Why?

Because `message` only exists inside `createMessage()`.

---

## 27. Block Scope

A block is code inside curly braces `{ }`.

Examples of blocks:

```
if (true) {  
  // block  
}  
  
for (let i = 0; i < 3; i++) {  
  // block  
}
```

Variables declared with `let` and `const` inside a block stay inside that block.

Example:

```
if (true) {  
  const score = 90;  
  console.log(score);  
}  
  
console.log(score);
```

Expected behavior:

```
90
ReferenceError
```

The first log works.

The second log fails because `score` exists only inside the `if` block.

---

## 28. Scope Beginner Rule

Keep variables in the smallest scope that needs them.

Weak style:

```
let total;

function calculateTotal(price, quantity) {
  total = price * quantity;
}
```

Cleaner style:

```
function calculateTotal(price, quantity) {
  const total = price * quantity;
  return total;
}
```

The cleaner version keeps `total` inside the function.

This reduces accidental changes from other parts of the program.

---

# Part J: Pure Functions

## 29. What Is a Pure Function?

A pure function has two main rules:

1. Same input gives the same output.
2. It does not change anything outside itself.

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3));  
console.log(add(2, 3));  
console.log(add(2, 3));
```

Expected output:

```
5  
5  
5
```

This is pure because the same inputs always give the same result.

---

## 30. Pure Function Example: Tax Calculation

```
function calculateTax(price, taxRate) {  
  return price * taxRate;  
}  
  
console.log(calculateTax(100, 0.1));  
console.log(calculateTax(100, 0.1));
```

Expected output:

```
10  
10
```

The function does not depend on hidden outside data.

It only uses its parameters.

---

## 31. Why Pure Functions Are Easier to Test

Pure functions are easier to test because they are predictable.

Example:

```
function isPassing(score) {  
  return score >= 70;  
}  
  
console.log(isPassing(85));  
console.log(isPassing(60));
```

Expected output:

```
true  
false
```

You can test it easily:

```
console.assert(isPassing(85) === true, "85 should pass");  
console.assert(isPassing(60) === false, "60 should not pass");
```

Pure functions are easier to reason about because they do not secretly change outside values.

---

## Part K: Side Effects

### 32. What Is a Side Effect?

A side effect happens when a function changes or uses something outside itself.

Examples of side effects:

- printing to the console
- changing a global variable
- changing the page
- showing an alert
- writing to browser storage
- making a network request

Example:

```
function showMessage(message) {  
  console.log(message);  
}
```

This function has a side effect because it prints to the console.

---

### 33. Side Effects Are Not Always Bad

Side effects are necessary in real web pages.

For example, a web app must sometimes:

- update page text
- respond to button clicks
- save data
- show messages

But side effects should be controlled.

Good beginner habit:

Keep calculation separate from side effects.

---

### 34. Separating Calculation from Side Effects

Less clean version:

```
function checkout(price, quantity) {  
  const total = price * quantity;  
  console.log(total);  
  return total;  
}
```

This function calculates and logs.

Cleaner version:

```
function getTotal(price, quantity) {  
  return price * quantity;  
}  
  
const total = getTotal(10, 3);  
console.log(total);
```

Now:

- `getTotal()` calculates
- `console.log()` displays

This makes the function easier to reuse and test.

---

## Part L: Callbacks and Higher-Order Functions

### 35. What Is a Callback?

A callback is a function passed into another function.

Simple meaning:

A callback is a function you give to another function so it can run later.

Example from browser events:

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

The arrow function is a callback.

It does not run immediately.

It runs later when the button is clicked.

---

### 36. Simple Callback Example

```
function runTask(callback) {  
  callback();  
}  
  
function sayHello() {  
  console.log("Hello");  
}  
  
runTask(sayHello);
```

Expected output:

```
Hello
```

Important detail:

```
runTask(sayHello);
```

passes the function.

This would call it immediately:

```
runTask(sayHello());
```

That is not the same thing.

---

## 37. What Is a Higher-Order Function?

A higher-order function is a function that does at least one of these:

1. Takes another function as an argument.
2. Returns another function.

Example that takes a function:

```
function runTask(callback) {  
  callback();  
}
```

Example that returns a function:

```
function createGreeting() {  
  return function() {  
    return "Hello";  
  };  
}
```

You will use higher-order functions more in later sprints when working with array methods like `forEach`, `map`, `filter`, and `reduce`.



For Sprint 5, understand the basic idea:

JavaScript functions can be treated like values.

---

## Part M: Function Factories

### 38. What Is a Function Factory?

A function factory is a function that creates and returns another function.

Example:

```
function multiplyBy(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}  
  
const double = multiplyBy(2);  
const triple = multiplyBy(3);  
  
console.log(double(5));  
console.log(triple(5));
```

Expected output:

```
10  
15
```

---

### 39. How the Function Factory Works

This line:

```
const double = multiplyBy(2);
```

creates a new function that remembers `factor = 2`.

Then this line:

```
console.log(double(5));
```

means:

```
5 * 2
```

This line:

```
const triple = multiplyBy(3);
```

creates a different function that remembers `factor = 3`.

Then:

```
console.log(triple(5));
```

means:

```
5 * 3
```

---

## Part N: Currying

### 40. What Is Currying?

Currying means breaking a function with multiple inputs into a chain of functions that each take one input.

Normal function:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3));
```

Curried version:

```
function add(a) {  
  return function(b) {  
    return a + b;  
  };  
}  
  
console.log(add(2)(3));
```

Expected output:

5

Simple meaning:

add(2)(3)

means:

first remember 2, then add 3

---

## 41. Beginner Currying Example

```
function createTaxCalculator(taxRate) {  
  return function(price) {  
    return price * taxRate;  
  };  
}  
  
const calculateTenPercentTax = createTaxCalculator(0.1);  
  
console.log(calculateTenPercentTax(100));  
console.log(calculateTenPercentTax(50));
```

Expected output:

10  
5

The returned function remembers the tax rate.

You only pass the price later.

Beginner note:

Do not worry about advanced currying patterns yet. For Sprint 5, understand that functions can return specialized functions.

---

## Part O: Closures

### 42. What Is a Closure?

A closure happens when an inner function remembers variables from the outer function where it was created.

Simple definition:

closure = function + remembered outer variables

Example:

```
function createCounter() {  
  let count = 0;  
  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
  
console.log(counter());  
console.log(counter());  
console.log(counter());
```

Expected output:

```
1  
2  
3
```

---

## 43. How the Closure Works

Inside `createCounter()`, this variable is created:

```
let count = 0;
```

Then `createCounter()` returns an inner function:

```
return function() {  
  count++;  
  return count;  
};
```

Normally, a local variable disappears after the function finishes.

But here, the inner function still remembers `count`.

That remembered access is the closure.

---

## 44. Private State with Closures

Closures can protect data from direct access.

Example:

```
function createCounter() {  
  let count = 0;  
  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();
```

```
console.log(counter());  
console.log(counter());
```

Expected output:

```
1  
2
```

This does not work:

```
console.log(count);
```

Why?

Because `count` is local to `createCounter()`.

Outside code cannot directly access it.

But `counter()` can still update it because of the closure.

Simple meaning:

The data is private, but the returned function can still use it.

---

## 45. Closure Example: Secret Message

```
function createSecret(secretPassword, secretMessage) {  
  return function(passwordAttempt) {  
    if (passwordAttempt === secretPassword) {  
      return secretMessage;  
    }  
  
    return "Access denied";  
  };  
}  
  
const getSecret = createSecret("js123", "Closures are useful");  
  
console.log(getSecret("wrong"));  
console.log(getSecret("js123"));
```

Expected output:

```
Access denied
Closures are useful
```

The returned function remembers:

- secretPassword
- secretMessage

That is a closure.

---

## Part P: Core Sprint 5 Code Pattern

### 46. Core Code Pattern

Type this code yourself.

Before running it, predict the output.

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}

console.log(greet("Anna"));
console.log(greet());

const add = (a, b) => a + b;
console.log(add(2, 3));

function multiplyBy(factor) {
  return function(number) {
    return number * factor;
  };
}

const double = multiplyBy(2);
console.log(double(5));
```

Expected output:

```
Hello, Anna!  
Hello, Guest!  
5  
10
```

---

## 47. Core Pattern Explanation

This function has a default parameter:

```
function greet(name = "Guest") {  
  return `Hello, ${name}!`;  
}
```

So this:

```
console.log(greet("Anna"));
```

prints:

```
Hello, Anna!
```

And this:

```
console.log(greet());
```

prints:

```
Hello, Guest!
```

This arrow function:

```
const add = (a, b) => a + b;
```

returns `a + b` automatically.

So:



```
console.log(add(2, 3));
```

prints:

```
5
```

This function factory:

```
function multiplyBy(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}
```

creates a new function.

This:

```
const double = multiplyBy(2);
```

creates a function that multiplies numbers by `2`.

So:

```
console.log(double(5));
```

prints:

```
10
```

---

## Part Q: Practical Sprint 5 Functions

### 48. Function: `cleanUsername(username)`

Goal:

Clean messy user input.

```
function cleanUsername(username) {  
  return username.trim().toLowerCase();  
}  
  
console.log(cleanUsername(" SANGEETH "));  
console.log(cleanUsername(" ADA_LOVELACE "));
```

Expected output:

```
sangeeth  
ada_lovelace
```

Concepts used:

- parameter
- string methods
- method chaining
- return value
- pure function

---

## 49. Function: `formatFullName(firstName, lastName)`

Goal:

Clean two names and combine them.

```
function formatFullName(firstName, lastName) {  
  const cleanFirst = firstName.trim();  
  const cleanLast = lastName.trim();  
  
  return `${cleanFirst} ${cleanLast}`;  
}  
  
console.log(formatFullName(" Ada ", " Lovelace "));
```

Expected output:

```
Ada Lovelace
```

Concepts used:

- multiple parameters
  - local variables
  - `trim()`
  - template literals
  - return value
- 

## 50. Function: `isPassing(score)`

Goal:

Return whether a score is passing.

```
function isPassing(score) {  
  return score >= 70;  
}  
  
console.log(isPassing(85));  
console.log(isPassing(60));
```

Expected output:

```
true  
false
```

Concepts used:

- parameter
  - comparison operator
  - boolean return value
  - pure function
- 

## 51. Function: `calculateTotal(price, quantity)`

Goal:

Calculate a shopping total.

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}  
  
console.log(calculateTotal(20, 3));  
console.log(calculateTotal(5, 4));
```

Expected output:

```
60  
20
```

Concepts used:

- numbers
- multiplication
- parameters
- arguments
- return value

---

## 52. Function: `calculateGrade(score)`

Goal:

Return a letter grade.

```
function calculateGrade(score) {  
  if (score >= 90) {  
    return "A";  
  }  
  
  if (score >= 80) {  
    return "B";  
  }  
  
  if (score >= 70) {  
    return "C";  
  }  
  
  return "Keep practicing";  
}
```

```
console.log(calculateGrade(95));  
console.log(calculateGrade(82));  
console.log(calculateGrade(65));
```

Expected output:

```
A  
B  
Keep practicing
```

Concepts used:

- conditions
- comparison operators
- multiple return statements
- function stops after return

---

## Part R: Mini-Build 1 — Shopping Cart Functions

### 53. Task

Create small functions for a simple shopping cart calculation.

```
function getSubtotal(price, quantity) {  
  return price * quantity;  
}  
  
function getTax(subtotal, taxRate) {  
  return subtotal * taxRate;  
}  
  
function getFinalTotal(subtotal, tax) {  
  return subtotal + tax;  
}  
  
const subtotal = getSubtotal(20, 3);  
const tax = getTax(subtotal, 0.1);  
const finalTotal = getFinalTotal(subtotal, tax);  
  
console.log(subtotal);
```

```
console.log(tax);  
console.log(finalTotal);
```

Expected output:

```
60  
6  
66
```

---

## 54. What This Mini-Build Teaches

This mini-build teaches that each function should have one clear job.

```
getSubtotal()
```

calculates the subtotal.

```
getTax()
```

calculates the tax.

```
getFinalTotal()
```

combines subtotal and tax.

This is better than putting everything into one large function.

---

## Part S: Mini-Build 2 — Counter with Closure

### 55. Task

Create a private counter.

```
function createCounter() {  
  let count = 0;  
  
  return function() {
```

```
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
  
console.log(counter());  
console.log(counter());  
console.log(counter());
```

Expected output:

```
1  
2  
3
```

---

## 56. What This Mini-Build Teaches

This teaches closure and private state.

The variable `count` is private.

Outside code cannot directly access it.

But the returned function can remember and update it.

This is the key closure idea.

---

# Part T: Mini-Build 3 — Password-Protected Secret

## 57. Task

Create a function that stores a secret message and only returns it when the correct password is given.

```
function createSecret(secretPassword, secretMessage) {  
  return function(passwordAttempt) {  
    if (passwordAttempt === secretPassword) {  
      return secretMessage;  
    }  
  }  
}
```

```
        return "Access denied";
    };
}

const getSecret = createSecret("js123", "Closures are useful");

console.log(getSecret("wrong"));
console.log(getSecret("js123"));
```

Expected output:

```
Access denied
Closures are useful
```

## 58. What This Mini-Build Teaches

This teaches that a closure can remember more than one outer variable.

The returned function remembers:

- `secretPassword`
- `secretMessage`

The outside code can call `getSecret()`.

But it cannot directly access the private variables inside `createSecret()`.

## Part U: Practice Set

### 59. Warm-Up Drill 1: Basic Function

Task:

Create a function that returns `"Hello"`.

```
function sayHello() {
    return "Hello";
}
```



```
console.log(sayHello());
```

Expected output:

```
Hello
```

---

## 60. Warm-Up Drill 2: Function with One Parameter

Task:

Create a function that greets a person by name.

```
function greet(name) {  
  return `Hello, ${name}`;  
}  
  
console.log(greet("Sangeeth"));
```

Expected output:

```
Hello, Sangeeth
```

---

## 61. Warm-Up Drill 3: Function with Two Parameters

Task:

Create a function that adds two numbers.

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10, 5));
```

Expected output:

## 62. Warm-Up Drill 4: Default Parameter

Task:

Create a greeting function with a default name.

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}  
  
console.log(greet("Ada"));  
console.log(greet());
```

Expected output:

```
Hello, Ada  
Hello, Guest
```

## 63. Warm-Up Drill 5: Arrow Function

Task:

Convert a normal add function into an arrow function.

Normal version:

```
function add(a, b) {  
  return a + b;  
}
```

Arrow version:

```
const add = (a, b) => a + b;  
  
console.log(add(2, 3));
```

Expected output:

5

---

## 64. Warm-Up Drill 6: Scope

Task:

Predict what happens.

```
function testScope() {  
  const message = "Inside function";  
  return message;  
}  
  
console.log(testScope());  
console.log(message);
```

Expected behavior:

Inside function  
ReferenceError

Explanation:

`message` only exists inside `testScope()`.

---

## 65. Warm-Up Drill 7: Pure Function

Task:

Create a pure function that checks if a number is even.

```
function isEven(number) {  
  return number % 2 === 0;  
}
```

```
console.log(isEven(8));  
console.log(isEven(7));
```

Expected output:

```
true  
false
```

---

## 66. Warm-Up Drill 8: Function Factory

Task:

Create a function that makes discount calculators.

```
function createDiscountCalculator(discountRate) {  
  return function(price) {  
    return price - price * discountRate;  
  };  
}  
  
const tenPercentOff = createDiscountCalculator(0.1);  
  
console.log(tenPercentOff(100));  
console.log(tenPercentOff(50));
```

Expected output:

```
90  
45
```

---

## Part V: Debug Challenges

### 67. Debug Challenge 1: Missing Return

Broken code:

```
function add(a, b) {  
  a + b;
```

```
}  
  
console.log(add(2, 3));
```

Expected output:

undefined

Why?

The function calculates `a + b`, but it does not return it.

Fixed version:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3));
```

Expected output:

5

---

## 68. Debug Challenge 2: Parameter vs Argument

Code:

```
function greet(name) {  
  return `Hello, ${name}`;  
}  
  
console.log(greet("Ada"));
```

Question:

Which is the parameter?

Answer:

name

Question:

Which is the argument?

Answer:

"Ada"

---

## 69. Debug Challenge 3: Local Variable Outside Function

Broken code:

```
function createScore() {  
  const score = 90;  
  return score;  
}  
  
console.log(score);
```

Problem:

`score` was created inside the function.

It does not exist outside the function.

Fixed version:

```
function createScore() {  
  const score = 90;  
  return score;  
}  
  
console.log(createScore());
```

Expected output:

90

## 70. Debug Challenge 4: Calling a Callback Too Early

Incorrect style:

```
function runTask(callback) {  
  callback();  
}  
  
function sayHello() {  
  console.log("Hello");  
}  
  
runTask(sayHello());
```

Problem:

`sayHello()` runs immediately.

Then its return value is passed into `runTask()`.

Correct version:

```
runTask(sayHello);
```

This passes the function itself.

## 71. Debug Challenge 5: Confusing `return` and `console.log()`

Code:

```
function add(a, b) {  
  console.log(a + b);  
}  
  
const result = add(2, 3);  
console.log(result);
```

Expected output:

```
5
undefined
```

Why?

The function prints `5`, but it does not return `5`.

So `result` becomes `undefined`.

Fixed version:

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
console.log(result);
```

Expected output:

```
5
```

---

## Part W: Common Sprint 5 Mistakes

### 72. Mistake 1: Forgetting to Call the Function

Wrong:

```
function greet() {
  return "Hello";
}

console.log(greet);
```

This prints the function itself, not the result.

Correct:



```
console.log(greet());
```

Use parentheses to call the function.

---

### 73. Mistake 2: Forgetting `return`

Wrong:

```
function multiply(a, b) {  
  a * b;  
}
```

Correct:

```
function multiply(a, b) {  
  return a * b;  
}
```

### 74. Mistake 3: Thinking `console.log()` Is the Same as `return`

Wrong idea:

```
console.log()
```

and

```
return
```

are the same.

Correct idea:

`console.log()` prints.

`return` gives a value back.

---

## 75. Mistake 4: Using a Local Variable Outside Its Scope

Wrong:

```
function test() {  
  const value = 10;  
}  
  
console.log(value);
```

Correct:

```
function test() {  
  const value = 10;  
  return value;  
}  
  
console.log(test());
```

---

## 76. Mistake 5: Doing Too Much in One Function

Weak version:

```
function checkout(price, quantity) {  
  const total = price * quantity;  
  console.log(total);  
  alert(total);  
  return total;  
}
```

Cleaner version:

```
function getTotal(price, quantity) {  
  return price * quantity;  
}  
  
const total = getTotal(10, 3);  
console.log(total);
```

One function should usually have one clear job.

---

## 77. Mistake 6: Changing Outside State Accidentally

Risky version:

```
let count = 0;

function addOne() {
  count++;
}

addOne();
console.log(count);
```

This changes a global variable.

Cleaner version when possible:

```
function addOne(count) {
  return count + 1;
}

console.log(addOne(0));
```

The cleaner version is more predictable.

---

## Part X: Mental Models

### 78. Function as a Machine

A function is like a machine.

```
input → machine → output
```

Example:

```
price and quantity → getTotal → total
```

---

## 79. Parameter as a Blank Space

A parameter is like a blank space in a sentence.

```
function greet(name) {  
  return `Hello, ${name}`;  
}
```

`name` is the blank.

When you call:

```
greet("Ada")
```

JavaScript fills the blank with `"Ada"`.

---

## 80. Return as the Answer

A function with `return` gives an answer back.

```
function add(a, b) {  
  return a + b;  
}
```

The returned value can be stored, printed, or used in another calculation.

---

## 81. Scope as a Boundary

Scope is like a boundary.

Variables inside the boundary cannot always be used outside it.

```
function test() {  
  const value = 10;  
}
```

`value` lives inside the function boundary.

---

## 82. Pure Function as a Predictable Machine

A pure function is predictable.

Same input always gives same output.

```
function double(number) {  
  return number * 2;  
}
```

`double(5)` always gives `10`.

---

## 83. Closure as Memory

A closure is a function with memory.

```
function createCounter() {  
  let count = 0;  
  
  return function() {  
    count++;  
    return count;  
  };  
}
```

The returned function remembers `count`.

---

# Part Y: Key Vocabulary

## 84. Vocabulary

### Function

Reusable block of code.

```
function greet() {  
  return "Hello";  
}
```

## Function Declaration

A named function created with the `function` keyword.

```
function add(a, b) {  
  return a + b;  
}
```

## Invocation / Function Call

Running a function with parentheses.

```
add(2, 3);
```

## Parameter

A placeholder in a function definition.

```
function greet(name) {  
  return name;  
}
```

## Argument

A real value passed into a function call.

```
greet("Ada");
```

## Return Value

The value a function sends back.

```
return a + b;
```

`undefined`

The value returned by a function when there is no `return` statement.

## Default Parameter

A fallback parameter value.

```
function greet(name = "Guest") {  
  return name;  
}
```

## Anonymous Function

A function without its own name.

```
const add = function(a, b) {  
  return a + b;  
};
```

## Function Expression

A function stored in a variable.

```
const add = function(a, b) {  
  return a + b;  
};
```

## Arrow Function

A shorter function syntax.

```
const add = (a, b) => a + b;
```

## Implicit Return

When an arrow function returns a value without writing `return`.

```
const double = number => number * 2;
```

## Scope

Where a variable can be used.

### Global Scope

Scope available broadly in the file or script.

## Local / Function Scope

Scope inside a function.

## Block Scope

Scope inside `{ }` blocks when using `let` and `const`.

## Pure Function

A function that gives the same output for the same input and does not change outside state.

## Side Effect

A change or action outside the function, such as logging, changing the page, or updating a global variable.

## Callback

A function passed into another function.

## Higher-Order Function

A function that takes another function or returns another function.

## Function Factory

A function that creates and returns another function.

## Currying

Breaking a function with multiple inputs into a chain of functions that each take one input.

## Closure

A function that remembers variables from the outer scope where it was created.

## Private State

Data that outside code cannot directly access, but a closure can still use.

---



# Part Z: Revision Questions

## 85. Questions

Try to answer these without looking at the notes.

1. What is the main goal of Sprint 5?
2. What is a function?
3. Why are functions useful?
4. What is the difference between defining a function and calling a function?
5. What symbol do you use to call a function?
6. What is a parameter?
7. What is an argument?
8. In `greet("Ada")`, what is the argument?
9. What does `return` do?
10. What does a function return if there is no `return` statement?
11. What is the difference between `return` and `console.log()`?
12. Why is `return` better than `console.log()` for reusable calculations?
13. What is a default parameter?
14. What is an anonymous function?
15. What is a function expression?
16. What is an arrow function?
17. What is implicit return?
18. What is scope?
19. What is global scope?
20. What is local scope?
21. What is block scope?
22. Why should variables stay in the smallest useful scope?
23. What is a pure function?
24. Why are pure functions easier to test?
25. What is a side effect?
26. Is `console.log()` a side effect?
27. Why should calculation and side effects often be separated?
28. What is a callback?
29. What is a higher-order function?
30. What is a function factory?
31. What is currying?
32. What is a closure?
33. Why can `counter()` remember `count`?
34. What does private state mean?
35. Why does `console.log(count)` fail outside `createCounter()`?
36. Why does forgetting `return` often produce `undefined`?
37. Why is doing too much in one function a weak habit?
38. How do closures help create private state?

# Final Sprint 5 Summary

Sprint 5 teaches you to organize logic into reusable functions.

The most important ideas are:

- A function is reusable code.
- Parameters are placeholders.
- Arguments are real values.
- `return` gives a value back.
- Without `return`, a function returns `undefined`.
- `console.log()` prints; it does not return a reusable value.
- Scope controls where variables can be used.
- Pure functions are predictable and easier to test.
- Side effects should be controlled.
- Functions can be passed around like values.
- A function can return another function.
- A closure is a function that remembers outer variables.
- Closures can create private state.

Main beginner target:

Write small functions that receive input, return output, and avoid unnecessary outside changes.